U.PORTO

FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

Design and Implementation of a Concurrent, Fault-Tolerant Chat Server

Parallel and Distributed Computing (L.EIC - FEUP)

CPD Project 2

Raul Nunes Andrade e Silva de Oliveira

Tiago Pinto Cardoso de Almeida

Tomás Freire Noronha Jardim

202303446, 202303450, 202306202

May 2026

Index

1. Introduction

This report details the design and implementation of a highly concurrent, fault-tolerant distributed chat system communicating over TCP. Developed strictly using Java SE 21+, the project adheres to rigorous architectural constraints: it operates entirely without external third-party libraries and explicitly avoids the use of pre-built thread-safe collections from the `java.util.concurrent` package.

To satisfy these constraints while maintaining high performance, the architecture leverages Java's modern Virtual Threads to manage lightweight, non-blocking client connections. Shared state is managed through custom-built thread-safe data structures utilizing `ReentrantReadWriteLock` mechanisms, ensuring data integrity without causing system bottlenecks. Furthermore, the server features a robust fault-tolerance protocol, utilizing time-expiring session tokens stored in persistent memory to allow clients to seamlessly auto-reconnect and resume their state following network interruptions or server crashes. Finally, the system extends standard chat functionality by integrating a localized AI assistant, interfacing directly with an Ollama Large Language Model (Llama 3) via native HTTP requests and custom JSON parsing.

The following sections will detail the overarching system architecture, the custom concurrency model, the fault-tolerance and recovery mechanisms, and the technical implementation of the AI bot integration.

2. System Architecture

2.1 High-Level Topology

The system operates on a centralized client-server architecture utilizing the Transmission Control Protocol (TCP). The core of the network is the `ChatServer`, which listens on a designated port for incoming client connections. Individual `ChatClient` instances establish long-lived TCP connections to this server, acting as thin clients that handle user input and interface rendering. All complex logic, state management, and message routing are strictly centralized on the server, ensuring a single source of truth for all connected users. Communication across the network is handled asynchronously via a text-based command protocol (e.g., `LOGIN`, `JOIN`, `MSG`).

2.2 Component Modularity

To ensure maintainability and separation of concerns, the server architecture is divided into specialized, decoupled managers:

- **AuthManager:** Handles the security layer, including user registration, credential validation, and the generation of secure UUID session tokens. It is also responsible for persisting this data to the local disk (`users.txt` and `tokens.txt`).
- **RoomManager:** Acts as the central router for the application. It safely tracks all active chat spaces and directs incoming client payloads to the correct destination.
- **ChatRoom & AIChatRoom:** `ChatRoom` encapsulates the state and broadcasting logic for a specific group of connected clients. `AIChatRoom` extends this base class, inheriting the standard broadcasting logic while adding an asynchronous bridge to communicate with a local Large Language Model via HTTP.

2.3 Decoupling Connection from Session

A defining architectural philosophy of this project is the strict separation of the physical network connection from the logical user session. A standard TCP socket is treated as a fragile, temporary communication pipe that is prone to network interruptions. By contrast, the user session is treated as a highly durable, persistent entity represented by a time-stamped UUID token.

Because of this decoupling, if a client's physical TCP connection drops, the server does not destroy the user's data. Instead, the `AuthManager` retains the session state in memory. When the client regains network access, it establishes a brand new TCP connection and presents its token. The server then instantly binds this new network pipe to the existing session, allowing the system to achieve robust fault tolerance without forcing the user to re-authenticate or manually navigate back to their previous room.

3. Concurrency & Threading Model

3.1 Lightweight Concurrency via Virtual Threads

To maximize server throughput and minimize operating system overhead, the application entirely avoids traditional, heavy OS threads. Instead, it leverages Java 21's Virtual Threads (`Thread.ofVirtual()`). When the `ChatServer` accepts a new TCP connection, it immediately hands the socket off to a dedicated virtual thread. Similarly, the `ChatClient` spawns a virtual thread exclusively to listen for incoming server messages, freeing the main client thread to block on user keyboard input. This architecture allows the system to theoretically handle thousands of concurrent users with a remarkably small memory footprint.

3.2 Custom Thread-Safe Data Structures

A primary constraint of this project was the strict prohibition of pre-built thread-safe collections, such as `ConcurrentHashMap`. To fulfill this requirement, thread safety was engineered manually using standard Java collections (e.g., `HashMap`, `ArrayList`) protected by `ReentrantReadWriteLock` mechanisms. This specific lock was chosen over standard `synchronized` blocks to optimize performance: it allows multiple threads to read data simultaneously (such as when broadcasting a message or listing available rooms) without blocking each other. The lock only enforces exclusive access during state mutations, such as when a user joins a room or a new session token is generated.

3.3 Asynchronous Non-Blocking Broadcasting

A critical vulnerability in traditional iterative chat servers is the "slow client" problem, where broadcasting a message to a room sequentially can cause the entire server to stall if one user has a poor network connection. To solve this, the `ChatRoom.broadcast()` method implements a highly concurrent scatter-gather approach. When a message is sent to a room, the server iterates through the list of connected clients and spawns a brand new, ephemeral virtual thread for every individual `writer.println()` execution. This ensures that the time it takes to deliver a message to a user with high latency has zero impact on the delivery speed for the rest of the room.

4. Authentication & Fault Tolerance

4.1 Token-Based Session Management & Persistence

To ensure secure and persistent user identities, the application completely avoids caching raw credentials on the client side. Instead, successful authentication generates a secure, randomized UUID session token. Both user credentials and active session tokens are persisted to the server's local disk (`users.txt` and `tokens.txt`) via the `AuthManager`. This file-based persistence guarantees that user accounts and active sessions are fully preserved even if the server process undergoes a hard crash and reboots.

To maintain security and prevent stale sessions from consuming memory indefinitely, tokens are strictly time-bound with a one-hour expiration limit. The `AuthManager` acts as a self-cleaning mechanism: whenever the server starts up or a client attempts to reconnect, the system verifies the token's timestamp against the current system clock. Expired tokens are actively rejected and purged from memory, forcing the client to safely re-authenticate.

4.2 Client-Side Auto-Recovery

The system is explicitly engineered to withstand fragile TCP connections. Within the `ChatClient`, the main network communication loop is wrapped in a dedicated fault-tolerance block. If the underlying socket drops and throws an `IOException`, the client application does not crash. Instead, it enters an automatic recovery loop, pausing for three seconds before attempting to re-establish a connection to the server. This retry mechanism continues until the server is back online, ensuring the application remains robust against temporary network outages.

4.3 Seamless State Resumption

A critical requirement of this architecture was ensuring that network disruptions do not interrupt the user experience or destroy the chat context. When the `ChatClient` successfully re-establishes a dropped TCP connection, it completely bypasses the standard login prompt. Instead, it instantly transmits a `RECONNECT` command along with its saved UUID token.

Once the server validates the token and binds the new TCP socket to the existing user session, the client automatically transmits a `JOIN` command using its locally cached `currentRoom` variable. From the perspective of the end user, this complex handshake is entirely invisible; their terminal simply states that the connection was temporarily lost, and they instantly resume sending and receiving messages in their previous chat room exactly where they left off.

5. AI Bot Integration

5.1 Polymorphic Room Design

To integrate artificial intelligence without disrupting standard user communication, the system utilizes object-oriented inheritance. The `AIChatRoom` class extends the base `ChatRoom` class, inheriting its custom thread-safe client list and broadcasting mechanisms. The `RoomManager` acts as a factory, inspecting the names of newly requested rooms; if a user types a command like `/join AI Study`, the manager detects the "AI" prefix and dynamically instantiates an `AIChatRoom` instead of a standard room. This polymorphic approach ensures that AI functionality is strictly sandboxed to designated areas while maintaining a unified interface for the server's routing logic.

5.2 Asynchronous LLM Communication

The AI integration relies on a local instance of the Ollama Large Language Model (specifically Llama 3) operating on port 11434. Because generating AI responses is a computationally expensive and time-consuming process, the `AIChatRoom` intercepts incoming user messages and appends them to a synchronized conversation history before spawning a new virtual thread to query the LLM. By offloading the HTTP request to an asynchronous virtual thread, the chat room remains entirely non-blocking. Human users can continue to send and receive messages in real-time without the server freezing while it waits for the AI model to finish generating its reply.

5.3 Zero-Dependency REST Implementation

A major constraint of this project was the prohibition of third-party libraries, meaning industry-standard JSON parsing tools like Gson or Jackson could not be used. To interface with the Ollama REST API, the system utilizes Java's native `java.net.http.HttpClient` to construct and transmit the JSON POST payload containing the model configuration and the room's message history. Upon receiving the HTTP response, the system utilizes a custom, manually engineered string manipulation method (`extractResponseValue()`). This algorithm programmatically scans the raw JSON string payload, locates the target `"response"` key, dynamically identifies the boundaries of the value, and unescapes formatting characters (such as newlines). This allows the server to successfully extract and broadcast the AI's response text while strictly adhering to the zero-dependency project requirements.

6. Conclusion

The development of this distributed chat server successfully demonstrates the implementation of a highly concurrent, fault-tolerant system using modern Java SE 21+ features. By strictly adhering to the project constraints—specifically the prohibition of external libraries and pre-built thread-safe collections—the architecture necessitated a deep, foundational approach to systems engineering.

Through the manual implementation of `ReentrantReadWriteLock` mechanisms and standard collections, the project proved that robust thread safety and shared state management can be achieved without relying on abstracted concurrent utilities. Furthermore, the integration of Java's Virtual Threads effectively resolved traditional concurrency bottlenecks, showcasing a highly scalable, non-blocking broadcast model.

The fault-tolerance requirements were successfully met by decoupling the physical TCP connections from logical user sessions. By implementing persistent, time-expiring UUID tokens, the system achieved seamless state recovery, allowing clients to survive network drops and server crashes without losing their chat context. Finally, the successful integration of a local Ollama LLM via native HTTP requests and custom JSON parsing highlighted the system's extensibility and adherence to zero-dependency design.

Ultimately, this project provided invaluable hands-on experience with low-level socket programming, manual synchronization, and the architectural patterns required to build resilient, distributed network applications.